PROJECT DOCUMENTATION · V1.0

Lead Finder Agent

Full-Stack Lead Generation & Business Intelligence Platform — Complete Technical Reference

Next.js 16 | FastAPI | MongoDB | TypeScript 5 | Python 3.12+ | Playwright | OpenAI | Tailwind CSS 4 | Docker Ready

25+	45+	100+	30+
API ROUTES	PYDANTIC MODELS	SERVICE MODULES	TEST FILES

CORE PURPOSE

- Automate lead generation from Google Maps
- AI-powered website analysis & scoring
- Multi-channel campaign management
- CRM pipeline with activity tracking
- Export reports in PDF, DOCX, XLSX, CSV

KEY TECHNOLOGIES

- Frontend — Next.js 16 + Tailwind CSS 4
- Backend — FastAPI + Python 3.12
- Database — MongoDB (Motor async)
- Scraping — Playwright + Chromium
- AI — OpenAI GPT + custom pipeline

DOCUMENT CONTENTS

- §1 — Project Overview & Tech Stack
- §2 — Architecture & Folder Structure
- §3 — Database Schema & Collections
- §4 — AI Pipeline & Background Workers
- §5 — Auth, Security & Frontend Arch
- §6 — Deployment Guide (DigitalOcean)
- §7 — Performance, Security & Maintenance

DEPLOYMENT STACK

- DigitalOcean Droplet (Ubuntu 22.04)
- NGINX Reverse Proxy + SSL/TLS
- Let's Encrypt (Certbot)
- Systemd Service Management
- MongoDB Atlas (Cloud DB)

Project Overview

Section 01

Purpose: Lead Finder Agent automates the entire lead generation lifecycle — scraping business directories, crawling websites for contact information, running AI-powered business intelligence analysis, and managing outreach through email & WhatsApp campaigns with a built-in CRM pipeline.

Discovery — Scraping

- Google Maps category search
- Geo-targeted scraping
- Smart deduplication
- Rating & review filters
- Directory crawling

AI Analysis — OpenAI

- Multi-stage analysis pipeline
- Lead scoring & prioritization
- SEO & performance audit
- Tech stack detection
- Sales talking points

CRM & Reports — Exports

- Pipeline stage management
- Activity tracking
- PDF / DOCX / XLSX export
- Automated daily reports
- Email & WhatsApp campaigns

Technology Stack

Section 02

Frontend Stack

COMPONENT	TECHNOLOGY	VERSION	PURPOSE
Framework	Next.js (App Router)	16	SSR / SSG / client-side routing
Language	TypeScript	5	Type-safe development
Styling	Tailwind CSS	4	Utility-first CSS framework
Icons	Lucide React	latest	Consistent SVG icon set
State	Zustand + persist	latest	Global auth & UI state
Data Fetching	TanStack React Query	v5	Server state, caching, refetch
UI Primitives	Radix UI	latest	Accessible headless components
Forms	React Hook Form + Zod	latest	Validation & form management
Animations	Framer Motion	latest	Smooth UI transitions
Notifications	Sonner	latest	Toast notifications

Backend Stack

COMPONENT	TECHNOLOGY	VERSION	PURPOSE
Framework	FastAPI	latest	Async REST API with auto-docs
Language	Python	3.12+	Core backend runtime
Database	MongoDB + Motor	async	Document store with async driver
Validation	Pydantic	v2	Data models & serialization
Scraping	Playwright	latest	Headless Chromium automation

COMPONENT	TECHNOLOGY	VERSION	PURPOSE
Background Jobs	APScheduler	latest	Cron-style scheduled tasks
Security	PyJWT + Passlib	latest	JWT auth + bcrypt hashing
Encryption	cryptography (Fernet)	latest	API key encryption at rest
Reports	ReportLab / WeasyPrint	latest	PDF & DOCX generation
AI	OpenAI + provider factory	latest	GPT-powered analysis

Project Structure

Section 03

Backend — `backend/`

DIRECTORY TREE

```
backend/
└── app/
    ├── api/
    │   ├── routes/          # 25+ modules
    │   │   ├── auth.py
    │   │   ├── leads.py
    │   │   ├── campaigns.py
    │   │   ├── ai.py
    │   │   ├── whatsapp.py
    │   │   ├── crm.py
    │   │   └── reports.py
    │   └── dependencies.py
    ├── core/
    │   ├── config.py
    │   ├── security.py
    │   ├── middleware.py
    │   └── state_machine/
    ├── db/
    │   ├── database.py
    │   └── init_db.py
    ├── models/          # 45 schemas
    ├── repositories/
    ├── services/        # 100+ modules
    │   ├── ai/
    │   ├── campaigns/
    │   ├── whatsapp/
    │   ├── reports/
    │   └── search_engine/
    ├── main.py
    └── websockets.py
├── tests/           # 30+ test files
└── scripts/         # 20+ utility scripts
```

Frontend — `frontend/`

DIRECTORY TREE

```
frontend/
└── app/
    ├── (dashboard)/  # Auth routes
    │   ├── dashboard/
    │   ├── leads/
    │   ├── search/
    │   ├── campaigns/
    │   ├── crm/
    │   ├── intelligence/
    │   ├── settings/
    │   └── admin/
    ├── login/
    ├── layout.tsx
    └── page.tsx
└── src/
    ├── components/
    │   ├── leads/
    │   └── settings/
    ├── features/
    │   └── auth/store.ts
    ├── lib/
    │   ├── axios.ts
    │   ├── queryClient.ts
    │   └── cn.ts
    └── types/
        └── index.ts
```

Frontend Routes

Route Path	Purpose
`/dashboard`	Main dashboard — real-time stats & KPI overview
`/dashboard/search`	Google Maps search interface with geo & category filters
`/dashboard/leads`	Lead management grid — sort, filter, tag, favourite
`/dashboard/campaigns`	Email & WhatsApp campaign builder & scheduler
`/dashboard/crm`	CRM Kanban pipeline — New → Qualified → Closed
`/dashboard/intelligence`	Business intelligence reports & aggregated data
`/dashboard/admin`	Admin panel — user management & system settings

`/login`	Authentication page with remember me + token refresh

Database Architecture

Section 04

MongoDB Collections

COLLECTION	PURPOSE
leads	Primary lead documents with nested AI analysis metadata
users	User accounts, password hashes, roles, and JWT sessions
campaigns	Email and WhatsApp campaign configurations
templates	Reusable email / WhatsApp message templates
search_sessions	Saved search configurations for quick re-run
ai_analysis_jobs	AI analysis job queue with status tracking
ai_analysis_queues	Queue management for AI workers
advanced_analysis_queue	Deep-dive complex analysis job queue
worker_nodes	Background worker health and status tracking
job_logs	Unified job execution logging
crm_activities	CRM activity log — calls, emails, meetings
intelligence_daily	Daily aggregate intelligence snapshots

Lead Document Schema

```
LeadDocument = { # ── Basic Business Info "name": str, "phone": str, "email": str, "website": str, "address": str, "city/state": str, "category": str, "google_rating": float, # ── AI Analysis (nested) "ai_analysis": { "analysis_status": WAITING|QUEUED|RUNNING|COMPLETED|FAILED, "overall_score": int, "seo_score": int, "performance_score":int, "recommendations": List[str], "emails": List[str], "technology_detected":List[str], }, # ── Enriched Data, CRM Fields, Metadata "pipeline_stage": str, "priority": str, "owner_id": str, "user_id": str, "tags": List[str], "is_favorite": bool, "created_at": datetime }
```

AI Analysis Pipeline

Section 05

Workflow State Machine

PENDING Queued → RUNNING Started → COLLECTING Crawl → ANALYZING AI Review → SCORING Calculate → SAVING MongoDB → ✓ SUCCESS Complete

Stage 1 — Collection

Playwright

- Crawl all website pages
- Discover & follow internal links
- Extract raw HTML content
- Capture contact emails & phones
- Detect social media profiles

Stage 2 — Analysis

OpenAI

- Full SEO metrics audit
- Page performance assessment
- UI/UX design quality audit
- Technology stack detection
- Opportunity gap identification

Stage 3 — Scoring

Weights

Stage 4 — Saving

MongoDB

- Overall business score (0–100)
- SEO score calculation
- Performance score weighting
- Priority classification
- Sales talking points generation

- Persist results to MongoDB
- Update lead document in-place
- Trigger enrichment queue
- Emit WebSocket progress event
- Mark job as SUCCESS / FAILED

Background Workers & Scheduler

Section 06

Startup Workers (auto-launched with backend)

CampaignQueueWorker
Email & WhatsApp campaign queue processor

email_worker_instance
Email dispatch queue handler

EnhancedWAQueueWorker
WhatsApp message dispatch queue

WADesktopQueueWorker
WhatsApp AI desktop automation

EmailAIQueueWorker
AI-powered email analysis processor

AIQueueWorker
Main AI analysis queue processor

AdvancedAnalysisWorker
Complex deep-dive analysis jobs

AutoReportWorker
Automated daily report generation

CRMAutomationWorker
CRM workflow automation engine

EnrichmentQueueWorker
Lead data enrichment queue

APScheduler Jobs

JOB	FREQUENCY	PURPOSE
emit_heartbeat	Every 30s	Worker health monitoring & WebSocket ping
daily_aggregate	Daily (midnight)	Aggregate daily intelligence snapshots
daily_cleanup	Daily (1 AM)	Data cleanup, log archival, old job purge

Authentication & Security

Section 07

Auth Flow — JWT

- Login with email + password
- JWT access token (15 min expiry)
- Refresh token (7 day expiry)
- localStorage / sessionStorage via "remember me"
- Auto token refresh on 401 errors
- Role-based access: Admin / User

Security Features — Hardened

- bcrypt password hashing (Passlib)
- Fernet encryption for API keys at rest
- CORS — environment-controlled origins
- Rate limiting via slowapi
- Security headers middleware
- Request size limit — 10MB max

Frontend State Architecture

Zustand Store

- useAuthStore with persist
- localStorage (remember me)
- sessionStorage (session only)

React Query

- Server state management
- Auto-caching & background refetch
- Stale-while-revalidate

Axios Client

- Request interceptors
- 401 auto token refresh
- Base URL from env vars

DigitalOcean Droplet Deployment

Section 08

Step 1 — Provision Droplet

SETTING	VALUE
Image	Ubuntu 22.04 LTS
Plan	$24/mo — 4 GB RAM, 4 vCPUs (minimum)
Authentication	SSH key (recommended)
Hostname	`lead-finder-server`
Datacenter	Choose region closest to your users

Step 2 — Initial Server Setup

BASH

```bash
# System update
apt update && apt upgrade -y

# Create non-root user
adduser yourusername
usermod -aG sudo yourusername

# Configure UFW firewall
ufw allow OpenSSH && ufw allow 22/tcp
ufw allow 80/tcp && ufw allow 443/tcp && ufw enable
```

Step 3 — Install Backend Dependencies

BASH

```bash
# Python 3.12
sudo apt install -y python3.12 python3.12-venv python3.12-dev build-essential

# MongoDB 7.0
curl -fsSL https://www.mongodb.org/static/pgp/server-7.0.asc | \
  sudo gpg --dearmor -o /usr/share/keyrings/mongodb-server-7.0.gpg
sudo apt install -y mongodb-org
sudo systemctl enable mongod && sudo systemctl start mongod

# Node.js 20.x
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# NGINX
sudo apt install -y nginx && sudo systemctl enable nginx
```

Step 4 — Backend Setup

BASH

```bash
cd ~/lead-finder-project/backend
python3.12 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip && pip install -r requirements.txt
pip install playwright && python -m playwright install chromium
```

```
# Generate secure keys
python -c "import secrets; print(secrets.token_urlsafe(32))"
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
```

Step 5 — Environment Variables (.env)

```
# APPLICATION
PROJECT_NAME="Lead Finder Agent API" API_V1_STR="/api/v1"

# DATABASE
MONGODB_URL=mongodb://localhost:27017/lead_finder_agent DATABASE_NAME=lead_finder_agent

# SECURITY
SECRET_KEY=your_generated_32char_secret_key_here ALGORITHM=HS256 ACCESS_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_MINUTES=10080

# ADMIN
ADMIN_EMAIL=admin@yourdomain.com ADMIN_PASSWORD=SecureAdminPassword123!

# SMTP
SMTP_SERVER=smtp.gmail.com # port 587 SMTP_USERNAME=youremail@gmail.com
```

Systemd Services & NGINX

Section 09

Backend Systemd Service

INI — /ETC/SYSTEMD/SYSTEM/LEAD-FINDER-BACKEND.SERVICE

```ini
[Unit]
Description=Lead Finder Backend API
After=network.target mongod.service

[Service]
Type=simple
User=yourusername
WorkingDirectory=/home/user/backend
ExecStart=.venv/bin/uvicorn app.main:app \
  --host 0.0.0.0 --port 8000 \
  --workers 4 --timeout-keep-alive 120
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Frontend Systemd Service

INI — /ETC/SYSTEMD/SYSTEM/LEAD-FINDER-FRONTEND.SERVICE

```ini
[Unit]
Description=Lead Finder Frontend
After=network.target

[Service]
Type=simple
User=yourusername
WorkingDirectory=/home/user/frontend
ExecStart=/usr/bin/node \
  --max-old-space-size=4096 \
  node_modules/.bin/next start -p 3000
Restart=always
Environment="NODE_ENV=production"
Environment="NEXT_PUBLIC_API_URL=https://api.yourdomain.com/api/v1"

[Install]
WantedBy=multi-user.target
```

NGINX Reverse Proxy Config

NGINX — /ETC/NGINX/SITES-AVAILABLE/LEAD-FINDER

```nginx
upstream frontend { server 127.0.0.1:3000; }
upstream backend  { server 127.0.0.1:8000; }

server {
  listen 80;
  server_name yourdomain.com www.yourdomain.com;
  return 301 https://$server_name$request_uri;
}

server {
  listen 443 ssl http2;
  server_name yourdomain.com www.yourdomain.com;
  ssl_certificate      /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
  ssl_certificate_key  /etc/letsencrypt/live/yourdomain.com/privkey.pem;

  location / { proxy_pass http://frontend; }
  location /api/v1/ {
    proxy_pass http://backend/api/v1/;
    proxy_read_timeout 86400; # WebSocket support
  }
}
```

SSL Certificate

BASH

```bash
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
sudo certbot renew --dry-run   # test auto-renewal
```

Strengths & Areas for Improvement

Section 10

✅ Key Strengths

- ✓ Scalable background worker architecture with task queues
- ✓ Comprehensive multi-stage AI analysis pipeline
- ✓ Production-ready circuit breakers & connection pooling
- ✓ Modular clean separation — API / Services / Models
- ✓ Real-time WebSocket progress tracking
- ✓ Full reporting suite — PDF, Excel, CSV, DOCX
- ✓ Integrated CRM with activity tracking
- ✓ Security-first — JWT, encryption, rate limiting

△ Areas for Improvement

- △ API documentation could be more comprehensive
- △ No visible frontend tests (Jest / Cypress)
- △ No CI/CD pipeline (GitHub Actions / GitLab CI)
- △ APM monitoring (Datadog / Sentry) not yet integrated
- △ Redis caching for sessions & frequent queries

Service Management Quick Reference

BASH — DAILY OPERATIONS

```bash
# Service control
sudo systemctl {start|stop|restart|status} lead-finder-backend
sudo systemctl {start|stop|restart|status} lead-finder-frontend

# View logs
sudo journalctl -u lead-finder-backend -f
sudo journalctl -u lead-finder-frontend -f
sudo tail -f /var/log/nginx/error.log

# Health check
curl http://localhost:8000/api/v1/health
curl https://yourdomain.com/api/v1/health
```